CocoConnect

Development Preparation Records

Group 38 · Prepared by: S.N. Vidanage (2023/IS/108)

1. Documents Reviewed Before Development

Document	Used for	Status
SRS Version 3	Baseline functional and non-functional requirements	**Done**
Requirement Analysis Document v2	Updated, prioritised requirements	**Done**
Functional Analysis Document v2	Screens, components, API endpoints and database interactions	**Done**
Technical Feasibility Document v1 and v2	Technology choices, build order and risks	**Done**
ER Diagram	Database entities and relationships	**Done**
Wireframes and design system	Screen layouts, colours and components	**Done**

2. Development Environment

Tool	Version / detail	Purpose	Status
Flutter SDK	3.44 (stable channel)	Mobile app	**Done**
Dart	3.12	Language for the app	**Done**
Node.js and npm	Node.js 24, npm 11	Backend runtime and packages	**Done**
Android Studio + Android SDK	With platform tools (adb)	Building and running the app	**Done**
Android emulator	Pixel 7, API 35	Testing the app	**Done**
Visual Studio Code	Flutter and Dart extensions	Code editor	**Done**
Git	2.55	Version control	**Done**
GitHub repository	CocoConnect (team repository)	Shared code	**Done**
Supabase	Hosted PostgreSQL project	Shared database	**Done**
Swagger UI	Served by the backend at /api-docs	API documentation and testing	**Done**

Check: flutter doctor reports no blocking issues; the emulator starts and the app installs.

3. Project Structure

```
CocoConnect/
  lib/                  Flutter app
    main.dart           entry point
    screens/            one file per screen (buyer/, orders/, chat/ ...)
    services/           API calls per area (auth, buyer, owner, chat)
    widgets/            reusable UI components
  assets/               images
  android/, ios/, web/  platform projects
  backend/              Node.js + Express API
    server.js           starts the server, mounts routes
    config/             database connection, Swagger
    routes/             URL definitions
    middleware/         login and role checks
    controllers/        business logic and SQL
    utils/              shared helpers
    sql/                numbered database scripts
    test/               automated tests
  pubspec.yaml          Flutter packages
```

4. Branching Plan

Branch	Purpose
main	Stable code and shared documentation
auth-senuji	Login and sign-up screens
buyer-senuji	Buyer marketplace screens
buyer-backend-senuji	Buyer and authentication API
integration-senuji	Integrated app and backend
Other members' branches	Estate owner, admin and other modules, merged into integration

5. Configuration

The backend reads its settings from a private file backend/.env, which is excluded from Git. A template (.env.example) lists the names only:

Variable	Meaning
PORT	Port the API runs on (5000)
DATABASE_URL	Connection string for the shared PostgreSQL database (kept private)
DATABASE_SSL	Use an encrypted database connection
JWT_SECRET	Secret used to sign login tokens (kept private)

- Secrets are shared privately within the team, never through GitHub or group chats.
- The app reaches the backend at 10.0.2.2:5000 from the Android emulator; another address can be set at run time.

6. Setup Steps

```
git clone <repository>
cd CocoConnect
git checkout integration-senuji


cd backend
npm install
# create backend/.env from .env.example
npm run dev                # http://localhost:5000/api-docs


cd ..
flutter pub get
flutter run                # with the Android emulator running
```

7. Conventions Agreed

- One screen per file; API calls only inside services, never directly in screens.
- Backend split into routes, middleware and controllers; each route documented in Swagger.
- Database changes only through numbered SQL scripts; existing team tables are extended, not replaced.
- Short, clear commit messages; push to your own branch, merge into the integration branch.
- Run flutter analyze and the backend tests before merging.

8. Readiness Checklist

Item	Status
Requirements and feasibility documents reviewed	**Done**
Tools installed and verified	**Done**
Repository cloned and branches created	**Done**
Project structure organised	**Done**

Item	Status
Backend configuration template in place	**Done**
Emulator runs the app	**Done**
Backend serves the API documentation	**Done**